

Application 1

Écrire un programme C multi-threadé dans lequel le père déclare une variable globale ***nbr_parts_gateau*** initialisée à 8. Il lance par la suite 4 threads dont chacun cherche à consommer un part chaque seconde. Après chaque consommation, le fils affiche le nombre de parts restants.

Dans le cas où le fils ne trouve plus de morceaux, il affiche le message suivant : « thread x : Gâteau terminé !!! », où x est le numéro du thread correspondant.

NB: Le père doit attendre la terminaison de tous ses fils.

```

#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
volatile int nbr_parts_gateaux=8;
void* manger (void* args)
{
    while (nbr_parts_gateaux > 0)
    {
        nbr_parts_gateaux--;
        printf ("thread%d : nombre de parts restants = %d\n", (int)(long)args, nbr_parts_gateaux);
        sleep(1);
    }
    printf ("thread%d : Gateau terminé\n", (int)(long)args);
    return 0;
}
int main()
{
    pthread_t tab[4];
    int i;
    for (i=0; i<4; i++)
        pthread_create(&tab[i], NULL, manger, (void*)(long) i);
    for (i=0; i<4; i++)
        pthread_join(tab[i], NULL); /*Eviter le risque de terminer le programme complètement
                                     sans avoir pu exécuter les threads */
    return 0;
}

```

La valeur sera toujours relue en mémoire à chaque accès et non stockée dans un registre temporaire

NULL pour utiliser taille de pile et priorité par défaut

NULL : pas besoin de retour du thread appelé

/*Eviter le risque de terminer le programme complètement sans avoir pu exécuter les threads */

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>
#include <unistd.h> // pour sleep()
volatile int nbr_parts_gateaux = 8; // variable partagée
pthread_mutex_t verrou; // déclaration du mutex

```

```

void* manger(void* args)
{
    int id = (int)(long)args; // conversion sûre
    while (1)
    {
        pthread_mutex_lock(&verrou); // Début section critique
        if (nbr_parts_gateaux > 0)
        {
            nbr_parts_gateaux--;
            printf("thread %d : nombre de parts restantes = %d\n", id, nbr_parts_gateaux);
            pthread_mutex_unlock(&verrou); // Fin section critique
            sleep(1); // simule le temps pour manger une part
        }
        else
        {
            pthread_mutex_unlock(&verrou); // Débloquent avant de sortir
            break; // plus de gâteau
        }
    }
    printf("thread %d : gâteau terminé\n", id);
    return 0;
}

```

```

int main()
{
    pthread_t tab[4];
    int i;
    pthread_mutex_init(&verrou, NULL); // initialisation du mutex

    for (i = 0; i < 4; i++)
        pthread_create(&tab[i], NULL, manger, (void*)(long)i);
    for (i = 0; i < 4; i++)
        pthread_join(tab[i], NULL);
    pthread_mutex_destroy(&verrou); // libération du mutex
    return 0;
}

```

Application 2

Écrire un programme C multi-threadé dans lequel le père déclare une variable globale **litres_eau** initialisée à 10. Il lance par la suite 3 threads, chacun représentant une pompe qui cherche à retirer un litre d'eau toutes les 2 secondes. Après chaque retrait, le thread affiche le nombre de litres restants dans le réservoir.

Dans le cas où le réservoir ne contient plus d'eau, le thread affiche le message suivant :

Pompe x : Réservoir vide !!!

où x est le numéro du thread correspondant.

NB: Le père doit attendre la terminaison de tous ses fils avant de terminer le programme.